

LESSON 5: LOOPS IN PYTHON & ITERATIVE ARCHITECTURE

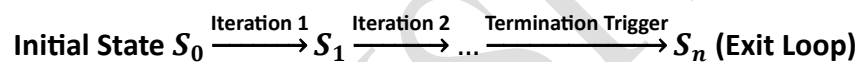
Python Programming Foundation Series — Comprehensive In-Depth Study Notes

1. FOUNDATION OF ITERATION & COMPUTATIONAL REPETITION

In software engineering, tasks rarely execute strictly once. Programs frequently process collections of records, poll remote network sockets, retry failed operations, or compute numerical series. Writing duplicate statements manually violates the DRY (Don't Repeat Yourself) principle, inflates bundle size, and degrades maintainability.

Loops are control-flow mechanisms designed to execute a block of code repeatedly based on either:

1. A Boolean Condition (while loop): Repetition continues as long as an invariant expression remains True.
2. Sequence Traversal (for loop): Repetition operates across the elements of a finite collection or stream until exhausted.



2. THE while LOOP: CONDITION-DRIVEN ITERATION

The while loop repeatedly evaluates a boolean expression before entering its body. If the condition evaluates to True, the indented suite executes; if it evaluates to False, execution jumps directly to the statement following the loop.

2.1 Formal Structure & Anatomy

A well-designed while loop consists of three distinct components:

- Initialization: Setting up an iteration state or sentinel variable before entering the loop.
- Evaluation: Testing the predicate expression prior to each iteration.
- State Mutation: Updating the state inside the loop body to ensure the loop progresses toward a termination condition.

Python

1. Initialization

```
counter = 1
```

2. Evaluation

```
while counter <= 5:
```

```
    print(f"Current iteration index: {counter}")
```

```
    # 3. State Mutation (Crucial: prevents infinite loops)
```

```
    counter += 1
```

2.2 The Hazard of Infinite Loops

If the condition never evaluates to False, the loop executes indefinitely, locking the CPU thread:

Python

```
# ❌ DANGEROUS: Infinite loop (Missing state mutation)
```

```
count = 1
```

```
while count <= 5:
```

```
    print(count)
```

```
    # Missing: count += 1 -> The loop never terminates!
```

3. THE for LOOP & THE PYTHON ITERATION PROTOCOL

Unlike traditional C-style for loops (which rely on loop counters like for (int i = 0; i < n; i++)), Python's for loop is a Collection-Based Iterator (similar to a for-each construct).

3.1 The Iteration Protocol Under the Hood

In Python, any object capable of returning its members one at a time is an Iterable (e.g., list, tuple, str, dict, set, range).

When you write:

Python

```
for item in collection:
```

```
    process(item)
```

Python executes the following steps behind the scenes:

1. Calls `iter(collection)` to obtain an iterator object implementing the `__next__()` method.
2. Calls `next(iterator)` on each loop iteration to retrieve the next value.

3. Automatically catches the `StopIteration` exception when items are exhausted and terminates the loop gracefully.

3.2 Iterating Over Collections

Python

1. String traversal (character-by-character)

```
for char in "Python":
```

```
    print(char, end=" ")
```

2. List traversal

```
frameworks = ["Django", "FastAPI", "Flask"]
```

```
for framework in frameworks:
```

```
    print(f"Backend Engine: {framework}")
```

3. Dictionary traversal (keys, values, and item tuples)

```
user_profile = {"id": 101, "username": "alex", "tier": "Enterprise"}
```

Default iteration yields keys

```
for key in user_profile:
```

```
    print(f"Key: {key} -> Value: {user_profile[key]}")
```

Iterating over key-value pairs directly via unpacking

```
for key, value in user_profile.items():
```

```
    print(f"{key}: {value}")
```

4. THE `range()` SEQUENCE GENERATOR

The `range()` function generates an immutable sequence of integers. In Python 3, `range` is not a list; it is an efficient generator-like sequence type that calculates numbers on demand in $O(1)$ memory, regardless of whether it spans 10 numbers or 10,000,000 numbers.

Syntax Variations

Syntax: `range(start = 0, stop, step = 1)`

(The start value is inclusive; the stop boundary is strictly exclusive).

Call Signature	Generated Values	Description
<code>range(5)</code>	0, 1, 2, 3, 4	Defaults start to 0, increments by +1
<code>range(2, 7)</code>	2, 3, 4, 5, 6	Explicit start at 2, stops before 7
<code>range(1, 10, 2)</code>	1, 3, 5, 7, 9	Skips by custom step +2
<code>range(10, 0, -2)</code>	10, 8, 6, 4, 2	Decrements backwards via negative step -2

5. LOOP CONTROL STATEMENTS: break, continue, AND pass

Control statements alter the normal execution sequence of loop iterations.

5.1 The break Statement

Instantly aborts the innermost enclosing loop. Execution resumes immediately at the statement following the loop block:

Python

```
for number in range(1, 10):
```

```
    if number == 5:
```

```
        print("Target value 5 located. Aborting loop.")
```

```
        break
```

```
    print(number, end=" ")
```

Output: 1 2 3 4 Target value 5 located. Aborting loop.

5.2 The continue Statement

Immediately halts execution of the current iteration and jumps back to the loop header to process the next element:

Python

```
for number in range(1, 6):  
    if number % 2 == 0:  
        continue # Bypasses even numbers  
    print(number, end=" ")
```

Output: 1 3 5

5.3 The pass Statement

A null operation (syntactic placeholder). Because Python enforces structural indentation, empty blocks raise a `SyntaxError`. `pass` satisfies the interpreter without executing any action:

Python

```
# Placeholder for future logic  
for item in heavy_stream_pipeline:  
    pass # Logic to be implemented in a later sprint
```

6. THE UNIQUE else CLAUSE WITH LOOPS

Python features a lesser-known, powerful syntactic capability: Loops can include an `else` block.

The Rule for loop-else:

- The `else` block executes if and only if the loop terminates normally (by exhausting its sequence or when the `while`-condition becomes `False`).
- If the loop is terminated prematurely by a `break` statement, the `else` suite is bypassed completely.

Python

```
target_id = 42  
data_records = [10, 25, 33, 42, 59]
```

```
for record in data_records:  
    if record == target_id:
```

```
print(f"Record {target_id} found!")
```

```
break
```

```
else:
```

```
# Executes ONLY if the loop never hit a 'break'
```

```
print(f"Record {target_id} does not exist in dataset.")
```

7. NESTED LOOPS & MULTI-DIMENSIONAL DATA STRUCTURES

A loop placed inside the body of another loop is called a nested loop. For every single iteration of the outer loop, the inner loop runs to full completion.

Computational Complexity Note

Nesting loops increases runtime complexity:

- Single loop over N items: $O(N)$ (Linear Time)
- Double nested loop over $N \times M$ items: $O(N \times M) \approx O(N^2)$ (Quadratic Time)
- Triple nested loop: $O(N^3)$ (Cubic Time)

Python

```
# 2D Grid / Matrix Processing
```

```
matrix = [
```

```
    [1, 2, 3],
```

```
    [4, 5, 6],
```

```
    [7, 8, 9]
```

```
]
```

```
for row_idx, row in enumerate(matrix):
```

```
    for col_idx, value in enumerate(row):
```

```
        print(f"Cell [{row_idx}][{col_idx}] = {value}", end=" | ")
```

```
    print()
```

8. COMPLETE PRODUCTION CODE SCRIPT

Below is an end-to-end Python script implementing iterative sequence traversal, step-based generation, mathematical algorithms, search routines, and multi-dimensional output:

Python

```
"""
```

Lesson 5: Loops in Python & Iterative Flow Architecture

Demonstrating while loops, for loops with range(), loop control directives, the loop-else clause, nested matrix operations, and classic algorithmic routines.

```
"""
```

```
# =====
```

1. THE WHILE LOOP: STATEFUL INVARIANT ITERATION

```
# =====
```

```
print("--- 1. While Loop Foundations ---")
```

```
accumulator = 0
```

```
iteration_pointer = 1
```

```
MAX_LIMIT = 5
```

```
while iteration_pointer <= MAX_LIMIT:
```

```
    accumulator += iteration_pointer
```

```
    print(f"Step {iteration_pointer}: Cumulative Sum = {accumulator}")
```

```
    iteration_pointer += 1
```

```
print(f"Final Computed Sum (1 to {MAX_LIMIT}) = {accumulator}")
```

```
# =====
```

2. THE FOR LOOP WITH RANGE() GENERATOR

```

# =====

print("\n--- 2. For Loop & Range Capabilities ---")

print("Ascending Step (1 to 10 by 2):")
for val in range(1, 11, 2):
    print(val, end=" ")
print()

print("Descending Step (10 down to 2 by -2):")
for val in range(10, 0, -2):
    print(val, end=" ")
print()

# Iteration with Enumerate
technologies = ["Python", "Docker", "Kubernetes", "PostgreSQL"]
print("\nInventory with Enumerate Index:")
for idx, tech in enumerate(technologies, start=1):
    print(f" Tier {idx}: {tech}")

# =====

# 3. CONTROL FLOW STATEMENTS: BREAK, CONTINUE, PASS

# =====

print("\n--- 3. Control Directives: Break & Continue ---")

# Skip evens (continue) and stop at ceiling limit (break)
print("Filter Odd Numbers (Abort at 7):")

```



```

for num in range(1, 15):
    if num % 2 == 0:
        continue # Skip even iteration
    if num > 7:
        print(f"[Ceiling {num} reached: Aborting]")
        break
    print(f"Active Odd Number: {num}")

# =====
# 4. SEARCH ALGORITHM USING THE FOR-ELSE CLAUSE
# =====

print("\n--- 4. Search Routine with Loop-Else Architecture ---")

def search_user_registry(target_username: str, registry: list):
    print(f"Initiating search query for: '{target_username}'...")
    for user in registry:
        if user == target_username:
            print(f"Match verified: '{user}' exists in registry.")
            break
    else:
        # Executes only if break was never encountered
        print(f"Exhausted database scan: '{target_username}' was not found.")

database_users = ["alice_dev", "bob_sysadmin", "charlie_qa"]
search_user_registry("bob_sysadmin", database_users)
search_user_registry("david_guest", database_users)

```

```
# =====
```

``` # 5. NESTED LOOPS: 2D MATRICES & PATTERNS ```

```
# =====
```

```
print("\n--- 5. Nested Loops: Tabular Multiplication & Matrix ---")
```

``` # Small 3x3 Multiplication Table ```

```
for row in range(1, 4):
```

```
    for col in range(1, 4):
```

```
        product = row * col
```

```
        print(f"{row}x{col}={product:<2}", end=" ")
```

```
    print()
```

``` # Inverted Pyramid Pattern ```

```
print("\nPyramid Projection:")
```

```
ROWS = 4
```

```
for i in range(1, ROWS + 1):
```

```
    print(" " * (ROWS - i) + "*" * (2 * i - 1))
```

```
# =====
```

``` # 6. PRACTICAL ALGORITHMS: FACTORIAL, PRIME, FIBONACCI ```

```
# =====
```

```
print("\n--- 6. Classical Algorithmic Case Studies ---")
```

``` # A. Factorial Computation: $N! = N * (N-1) * \dots * 1$ ```

```
target_n = 5
```

```
factorial_total = 1

for factor in range(1, target_n + 1):

    factorial_total *= factor

print(f"Computed Factorial of {target_n}! = {factorial_total}")
```

B. Deterministic Prime Number Verification

```
test_candidate = 29
```

```
is_prime = True
```

```
if test_candidate <= 1:
```

```
    is_prime = False
```

```
else:
```

```
    # Test divisors up to the square root of the number: int(n**0.5) + 1
```

```
    for divisor in range(2, int(test_candidate ** 0.5) + 1):
```

```
        if test_candidate % divisor == 0:
```

```
            is_prime = False
```

```
            break
```

```
print(f"Prime Assessment for {test_candidate}: {'PRIME' if is_prime else 'COMPOSITE'}")
```

C. Fibonacci Sequence Generation (Iterative State Swap)

```
fib_terms = 7
```

```
fib_a, fib_b = 0, 1
```

```
fib_output = []
```

```
for _ in range(fib_terms):
```

```
fib_output.append(fib_a)
```

```
fib_a, fib_b = fib_b, fib_a + fib_b
```

```
print(f"First {fib_terms} terms of Fibonacci series: {fib_output}")
```

9. CODE ARCHITECTURE BREAKDOWN: STEP-BY-STEP FLOW

Step 1: Stateful Mutation in the while Accumulator

The accumulator starts at 0 and `iteration_pointer` at 1. Before entering the loop block, Python checks the relational predicate `iteration_pointer <= MAX_LIMIT` ($1 \leq 5 \rightarrow \text{True}$).

- Iteration 1 adds 1 (*accumulator* = 1) and increments the pointer to 2.
- This sequence repeats until the pointer increments to 6.
- At the start of iteration 6, $6 \leq 5$ evaluates to False, immediately terminating the loop.

Step 2: Constant-Memory Evaluation with range()

When Python runs `range(1, 11, 2)`, it does not allocate an eleven-integer array in physical memory. It initializes an internal range structure that calculates the next integer algebraically ($start + step \times i$) only when `next()` is requested by the iterator protocol, preserving memory.

Step 3: Loop-Else Search Dynamics

In `search_user_registry`, when searching for "bob_sysadmin":

1. Iteration 1 evaluates `"alice_dev" == "bob_sysadmin"` (False).
2. Iteration 2 evaluates `"bob_sysadmin" == "bob_sysadmin"` (True).
3. The `break` statement fires. The interpreter halts iteration and skips the `else: block`.
4. When searching for "david_guest", no element triggers `break`. Upon exhausting the list, the interpreter falls into the `else: block`, outputting the missing-record alert.

10. JUNIOR ANTI-PATTERNS & PERFORMANCE PITFALLS

Anti-Pattern 1: Mutating a Collection While Iterating Over It

Python

❌ CATASTROPHIC BUG: Modifying list during traversal skips elements!

```
numbers = [1, 2, 2, 3, 4]
```

```
for num in numbers:
```

```
    if num == 2:
```

```
        numbers.remove(num) # Modifies underlying list indices dynamically!
```

Result: numbers becomes [1, 2, 3, 4] -> A '2' is missed completely!

 **CORRECT:** Iterate over a shallow slice copy or use a comprehension

```
numbers = [1, 2, 2, 3, 4]
```

```
numbers = [num for num in numbers if num != 2] # Clean, non-mutating filter
```

Anti-Pattern 2: The range(len(...)) Code Smell

Python

 **UN-PYTHONIC:** Simulating C-style index iteration

```
items = ["TokenA", "TokenB", "TokenC"]
```

```
for i in range(len(items)):
```

```
    print(i, items[i])
```

 **IDIOMATIC PYTHON:** Use enumerate() directly

```
for i, item in enumerate(items):
```

```
    print(i, item)
```

Anti-Pattern 3: Unnecessary Full Primality Traversal

Python

 **INEFFICIENT:** Checks every single integer up to N (O(N) runtime)

```
for i in range(2, N):
```

```
    if N % i == 0: ...
```

 **OPTIMIZED:** Check divisors only up to $\sqrt{N}$ ($O(\sqrt{N})$ runtime)

```
for i in range(2, int(N ** 0.5) + 1):
```

```
    if N % i == 0: ...
```

11. TECHNICAL INTERVIEW & RAPID RECAP CHEAT SHEET

High-Yield Questions & Answers

Q1: How does a for loop work internally in Python?

Answer: Python invokes the object's `__iter__()` method to obtain an iterator instance. On every iteration, it calls the iterator's `__next__()` method. When no items remain, `__next__()` raises a `StopIteration` exception, which the for loop intercepts internally to terminate cleanly without crashing.

Q2: What is the exact execution condition for a loop's else block?

Answer: A loop's else block executes when the loop finishes naturally—either by exhausting all items in a for loop or when the condition in a while loop evaluates to False. If the loop exits early via a break statement, the else block is skipped.

Q3: What is the memory footprint difference between `range(1000000)` and `list(range(1000000))`?

Answer: `range(1_000_000)` allocates an immutable range object of constant size ($O(1)$ memory, typically ≈ 48 bytes) that generates values on the fly. In contrast, `list(range(1_000_000))` allocates physical memory for one million individual integer pointers ($O(N)$ memory, consuming roughly ≈ 8 megabytes of RAM).

Q4: When should you use a while loop instead of a for loop?

Answer: Use a for loop when iterating over a known sequence, collection, or predefined number of cycles. Use a while loop when iteration depends on an indeterminate condition, event, or sentinel value (e.g., reading socket buffers until an EOF byte, checking hardware sensor states, or validating interactive user input until correct).